

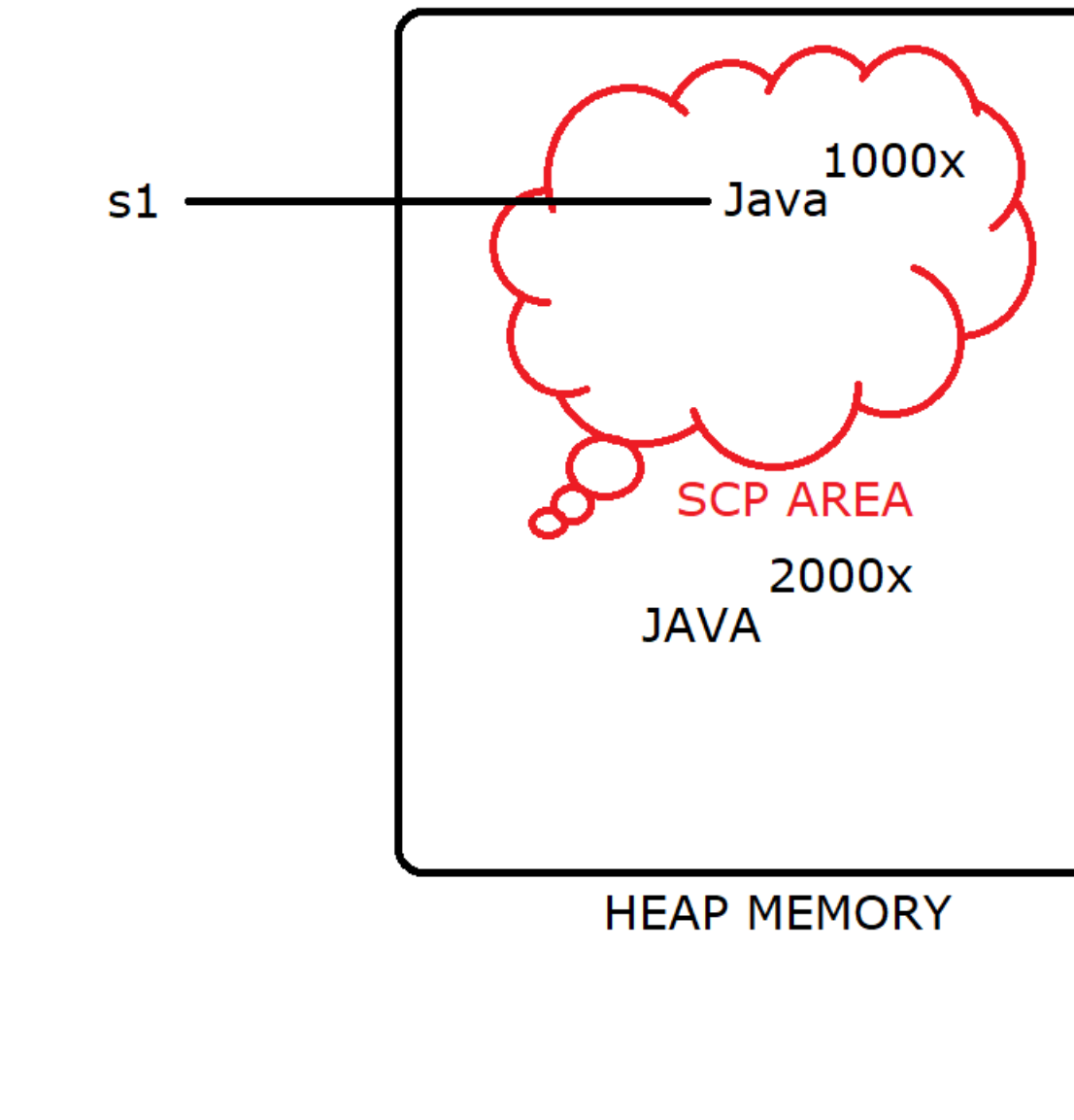
changes in String memory from JDK 1.8V with MetaSpace memory :

Upto JDK 1.7V, HEAP memory was the part of **PermGen (Permanent Generation memory)** which was a **fixed memory** means we cannot expand the size of HEAP but from JDK 1.8V onwards we have new memory called **Metaspace** which is dynamically growable memory.

* With Metaspace now we can expand the size of HEAP memory to accomdate more elements inside the existing HEAP memory.

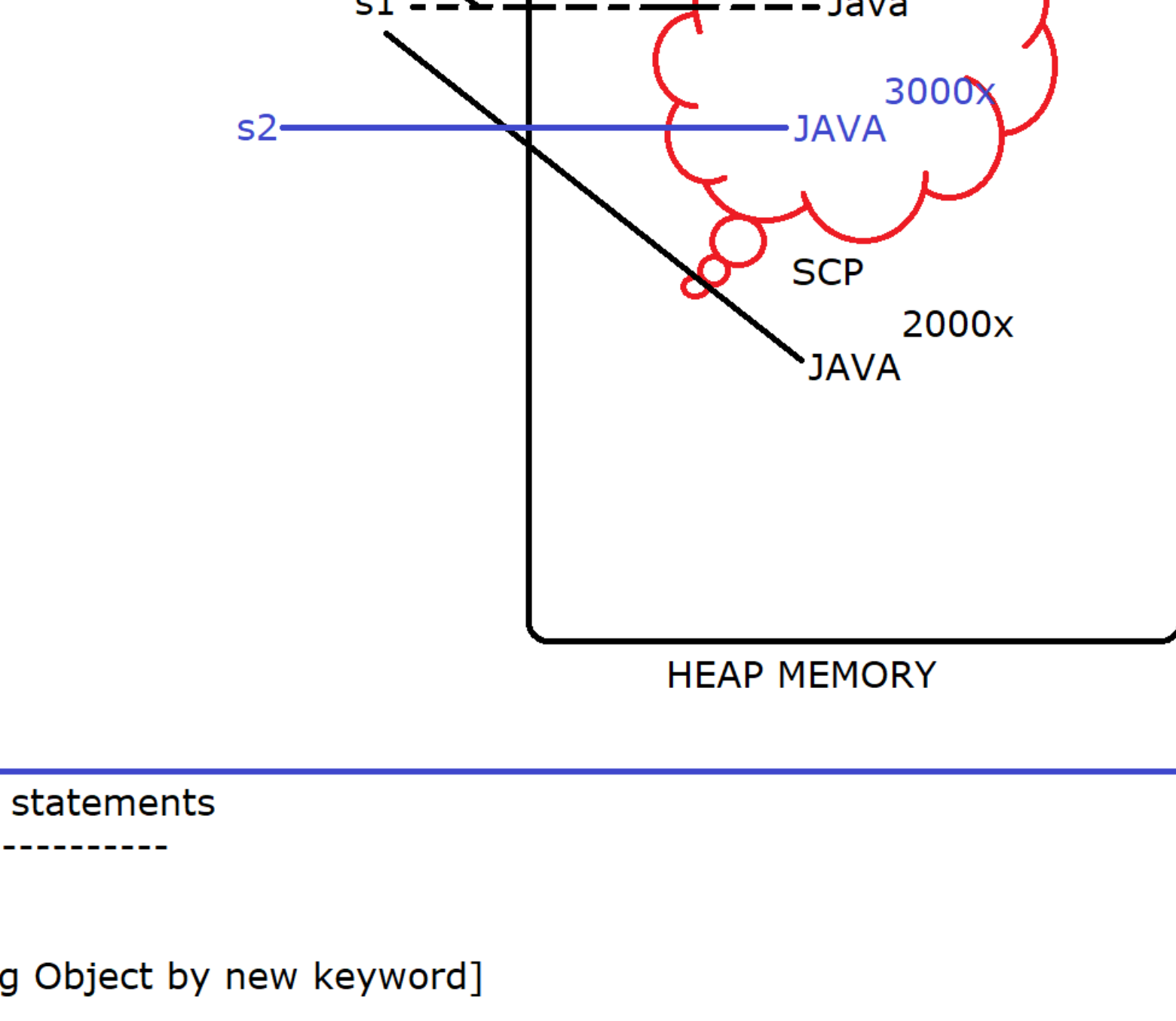
Memory Diagram changes :

```
String s1 = "Java";
s1.toUpperCase();
IO.println(s1);
```



Practical Proof

```
1) String s1 = "Java";
2) s1 = s1.toUpperCase();
3) String s2 = "JAVA";
4) IO.println(s1==s2); //false
```



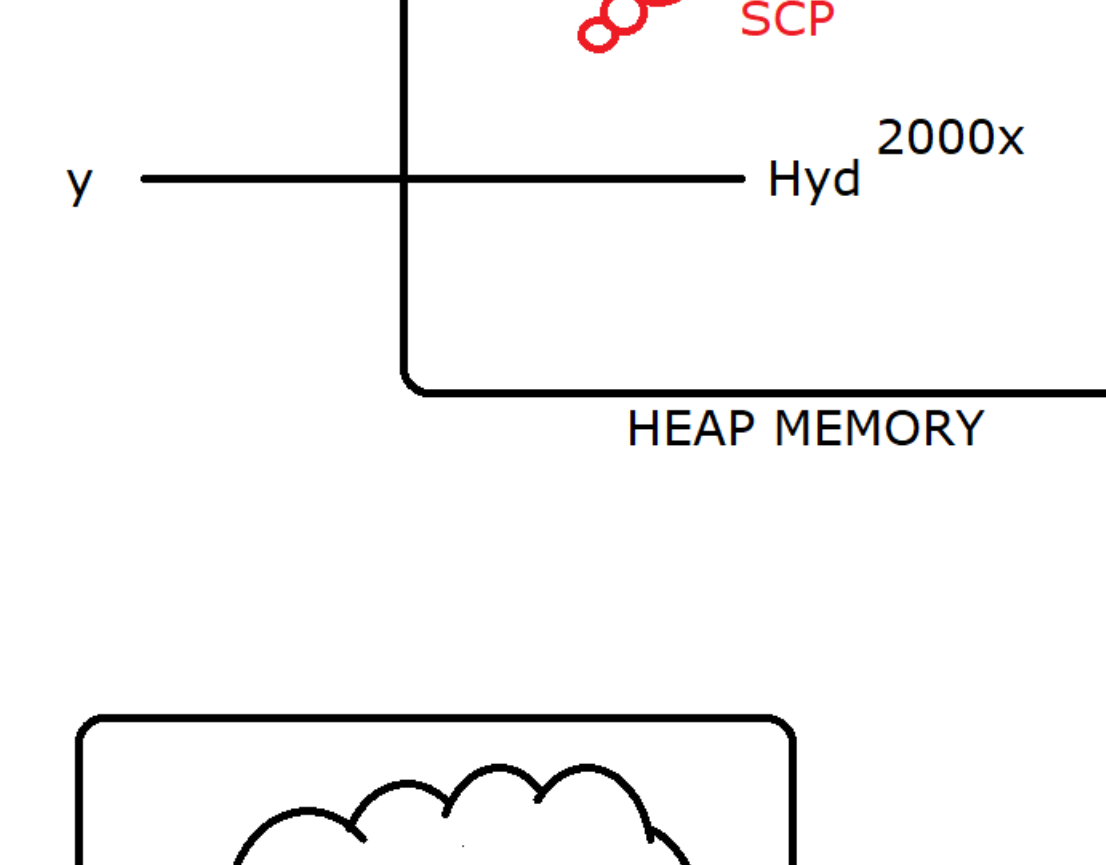
***What is the difference between the following two statements

String x = "Naresh"; [Creating String by Literal]

String y = new String("Hyderabad"); [Creating String Object by new keyword]

Case 1 :

```
String x = "Naresh";
String y = new String("Hyd");
```



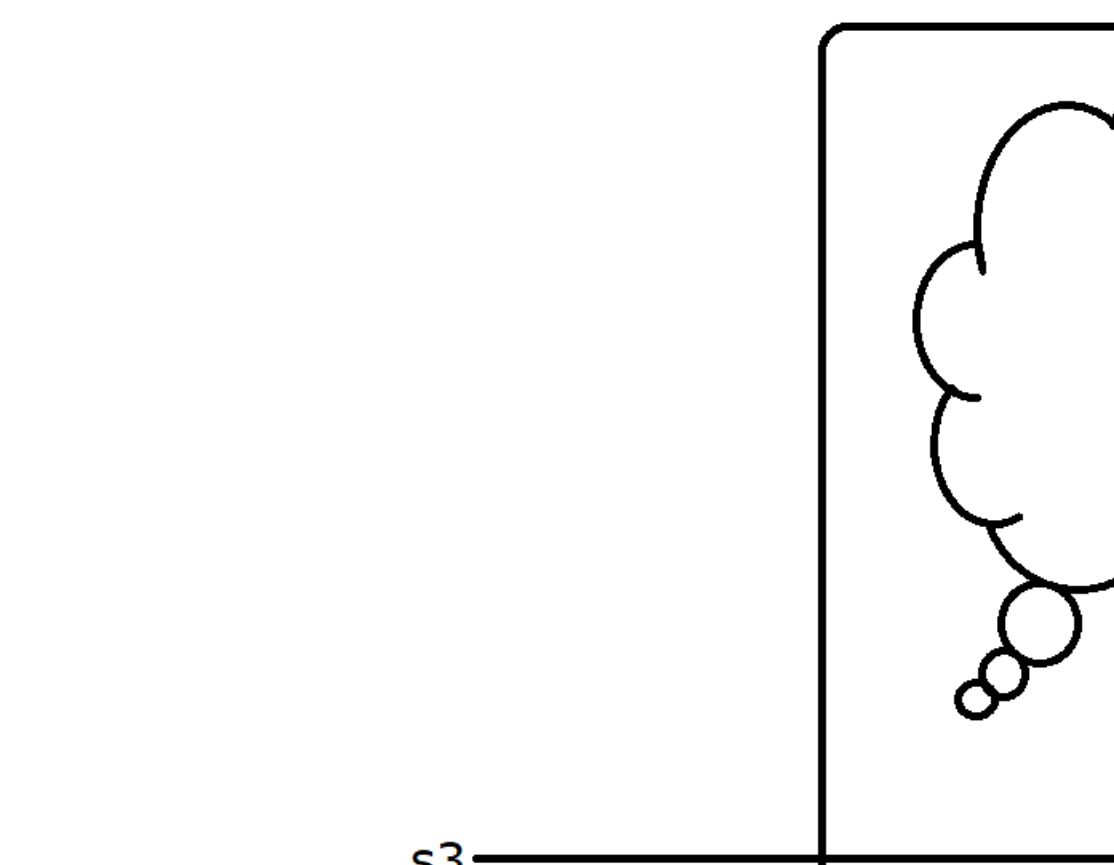
Case 2 :

```
String s1 = "Java";
String s2 = new String("Java");
```

[In this case using new keyword only one object will be created]

Practical Proof :

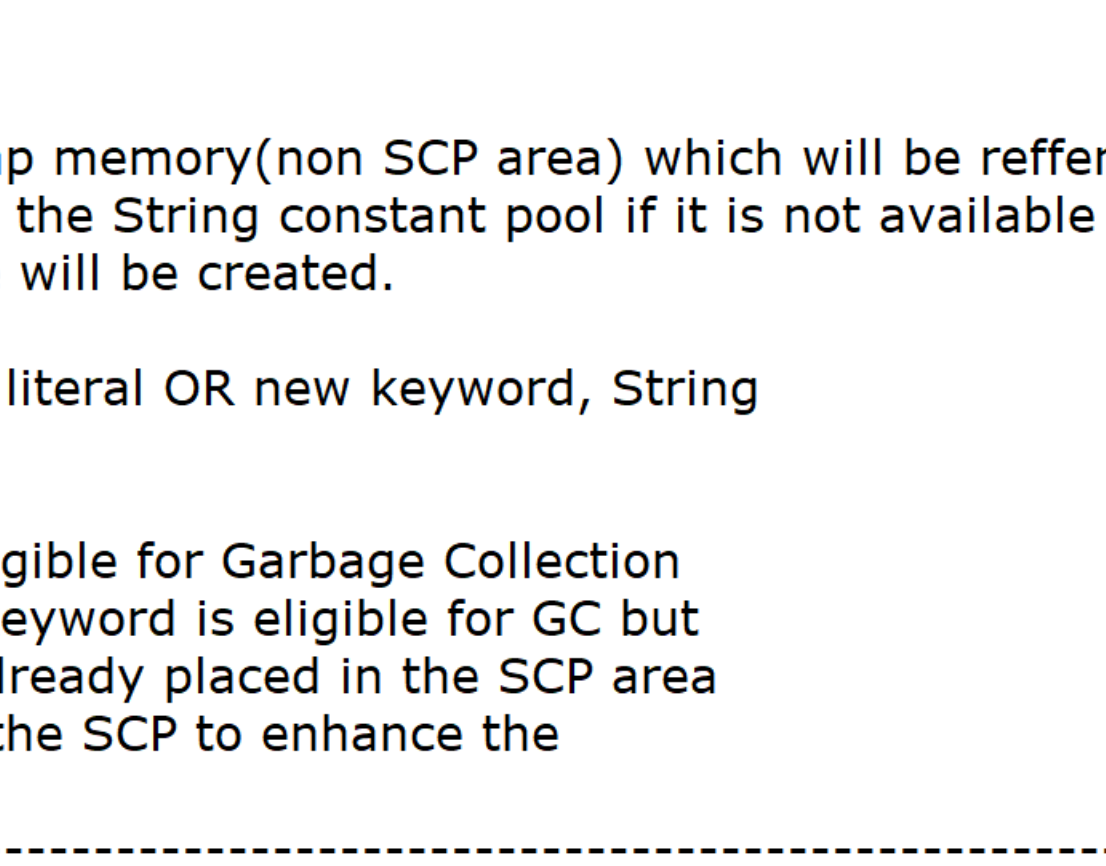
```
void main()
{
    String s1 = "Java";
    String s2 = new String("Java");
    IO.println(s1==s2); //false
}
```



Case 3 :

```
String s3 = new String("India");
String s4 = new String("India");
```

IO.println(s3==s4); //false



String x = "Naresh";

It will create one String object and one reference variable and String object will be created in the String constant pool.

String y = new String("Hyderabad");

It will create two String objects one is inside the heap memory(non SCP area) which will be referred by y reference variable and the same String object will be placed in the String constant pool if it is not available there. Hence two String Objects and one reference variable will be created.

Note :- 1) Whether we create String object by using literal OR new keyword, String objects are immutable.

2) String which is available in SCP area is not eligible for Garbage Collection but the String which is created by using new keyword is eligible for GC but It ensures that new keyword String object is already placed in the SCP area using intern() method, If it is not available in the SCP to enhance the reusability of the String.

public String intern() :

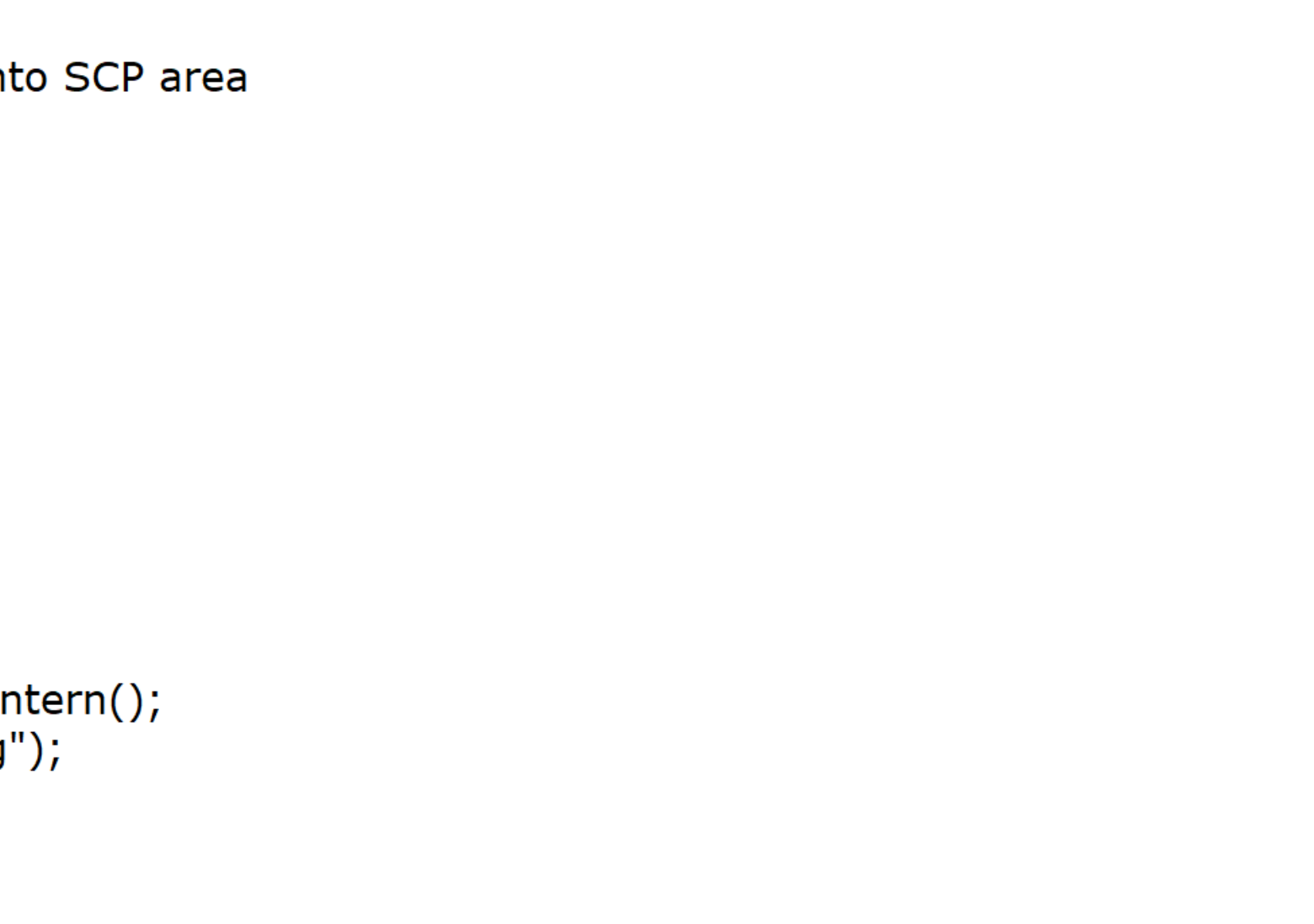
"Hello"; -> JVM will place the String in the SCP area
-> Internally JVM will invoke intern() method

intern.java

```
void main()
{
    String s1 = "Java";
    String s2 = new String("Java");

    s2 = s2.intern();

    IO.println(s1==s2);//true
}
```



It returns the canonical representation for the string object.

When we call intern() on a String object created by using new keyword, It will place the object in the SCP area (if not available in the SCP) and returns the reference from SCP.

If the String is already existing in the SCP area then It will **return the reference of the existing String object** from the SCP.

Note :- JVM Internally uses intern process to place the String literal ("") inside SCP area [interning process]

Intern.java

```
void main()
{
    String s1 = "Java";
    String s2 = new String("Java");

    s2 = s2.intern();

    IO.println(s1==s2);//true
}
```

How many ways we can put the String object into SCP area :

The following are the ways to put the String object into SCP area

- 1) By using Literal
- 2) By using intern() method of new Object
- 3) By using valueOf() static method of String class.
- 4) By using '+' operator [Compile time constant]

SCPStorage.java

```
void main()
{
    String s1 = "James Gosling";
    String s2 = new String("James Gosling").intern();
    String s3 = String.valueOf("James Gosling");
    String s4 = "James" + " Gosling";

    System.out.println(s1==s2); //true
    System.out.println(s1==s3); //true
    System.out.println(s1==s4); //true
}
```